

Theory Summary

Linear Structures & Trees, in one set of notes

Every theory slide from the two companion decks, condensed topic-wise into definitions, rules, formulas and complexity. No exercises, no practice-bank questions — reference material only.

15

topic sections

2

source decks, fully covered

0

exercises — theory only

WHAT THIS COVERS

Part A — Linear structures: Fundamentals & Big-O · Arrays · Linked Lists & Skip Lists · Stacks · Queues & Priority Queues, plus a compare-and-review cheat sheet.

Part B — Trees: Tree basics & representations · Binary trees & traversals · Binary Search Trees · AVL trees · Red-Black trees · Splay trees, plus a second compare-and-review cheat sheet.

Companion to the syllabus-filtered Previous-Year Question Bank built from the same two decks.

Contents

PART A · LINEAR STRUCTURES

1. Fundamentals	page 3
2. Arrays	page 4
3. Linked Lists	page 5
4. Stacks	page 6
5. Queues	page 7
6. Linear structures — cheat sheet	page 8

PART B · TREES

7. Tree basics	page 9
8. Binary trees & traversals	page 10
9. Binary Search Trees	page 11
10. AVL trees I — balance & rotations	page 12
11. AVL trees II — insert, delete & summary	page 13
12. Red-Black trees I — rules & insertion	page 14
13. Red-Black trees II — deletion & summary	page 15
14. Splay trees	page 16
15. Trees — compare & review cheat sheet	page 17

Fundamentals

What is a data structure?

A mathematical / logical way of organising data in memory so it can be used effectively. It bundles (1) how the data sits in memory and (2) the operations allowed on it (Handbook §2.1).

Abstract Data Type (ADT) vs data structure

- An ADT is a specification: what operations are allowed (e.g. push, pop) and what they mean.
- A data structure is an implementation of an ADT (e.g. the Stack ADT via an array or a linked list).
- Same ADT, several possible data structures — each with its own time/space trade-off.

What we judge a structure by (Handbook §2.1)

TIME	SPACE	SIMPLICITY
search/insert/delete speed	extra memory used	ease of correct implementation

Classifying data structures

LINEAR — one predecessor, one successor

Array · Linked List · Stack · Queue. Simple, fast sequential access, easy to traverse in order. This deck covers all four in depth.

NON-LINEAR — branches to more than 2 neighbours

Tree · Graph · Heap · Trie. Hierarchies and networks; fast priority access. Covered later: Trees, and self-balancing BSTs (AVL, Red-Black, Splay).

Measuring cost: Big-O recap

Notation	Feel of the growth
$O(1)$	constant — same cost regardless of n
$O(\log n)$	grows very slowly (binary search)
$O(n)$	grows in proportion to n (linear search)
$O(n \log n)$	typical of good sorting algorithms
$O(n^2)$	grows fast — watch nested loops over the same data

THE BIG PICTURE

The same data, stored differently, can make an algorithm run in $O(1)$, $O(\log n)$ or $O(n)$ time for the same task. Choosing a data structure is a trade-off: speed of search vs. speed of insertion vs. memory used vs. simplicity of code — every topic that follows is one more way to make that trade.

Arrays

Memory model

An array is a collection of (index, value) pairs stored contiguously — one straight block of equal-sized cells. Any element is reachable in $O(1)$ by its index.

$\text{address}(A[i]) = \text{base} + i \times (\text{size of one element})$
 e.g. $\text{base}=1000$, 4-byte ints: $\text{address}(A[3]) = 1000 + 3 \times 4 = 1012$

2D arrays: row-major vs column-major

A 2D array is a 1D array in disguise — the formula maps two indices onto one contiguous block.

Layout	Address formula	Used by
Row-major	$\text{base} + w \times (i \times \text{cols} + j)$	C, C++, Java — store row 0 fully, then row 1 ...
Column-major	$\text{base} + w \times (j \times \text{rows} + i)$	Fortran, MATLAB — store column 0 fully, then col 1 ...

Static vs dynamic

- Static: size fixed at compile time (`int list[20]`).
- Dynamic: size decided — and changeable — at run time (`int* list = new int[25]`).
- C++ does no auto range-checking; the programmer tracks the size.

Operations & their cost

Operation	Cost	Why
Retrieve by index	$O(1)$	direct address-formula lookup
Linear search	$O(n)$	scan until match or end
Binary search	$O(\log n)$	array must be sorted first
Insert / delete (middle)	$O(n)$	later elements must shift
Insert / delete (at end)	$O(1)^*$	*only if spare capacity exists

Dynamic (growable) arrays

Starts with some capacity; when full, allocate a new array (commonly double the size), copy everything across, free the old array. A single growth step costs $O(n)$ — but it happens rarely, so n insertions cost $O(n)$ total: $O(1)$ amortized per insert. (This is how C++ `std::vector`, Java `ArrayList` and Python list work internally.)

Strengths	Weaknesses
$O(1)$ random access · cache-friendly · compact, no pointer overhead	Fixed size (static) · $O(n)$ mid-array insert/delete · resizing costs $O(n)$

Linked Lists

The node, and the core trade-off

A node stores data plus a pointer to the next node (Handbook §2.3). Nodes need not be contiguous — each can live anywhere; the pointer is what links them. The list object keeps just one pointer, first, to the head node; the last node's pointer is NULL.

	Array	Linked list
Access by index	$O(1)$	$O(n)$ — walk from first
Insert/delete at front	$O(n)$ — shift	$O(1)$
Insert/delete at known node	$O(n)$ — shift	$O(1)$
Search	$O(n)$, or $O(\log n)$ sorted	$O(n)$ always — no binary search
Extra memory / element	none	one (or two) pointers

Insertion & deletion

- Insertion: 4 spots — front, after a node, before a node, at the end. Front is $O(1)$; the others are $O(1)$ once the target node is known (finding it by value costs $O(n)$).
- Deletion: needs a pointer to the predecessor — set `predecessor.next = current.next`, then free `current`. Both ops are $O(1)$ once the needed pointers are in hand; "delete key k " overall costs $\Theta(n)$ because you must search first.
- Sorted lists: insertion becomes $O(n)$ (must walk to the right spot) and — unlike a sorted array — still cannot be binary-searched. What they ARE good at: merging two sorted lists in $O(n+m)$, no extra space.

Four flavours

Type	Traversal	Best for
Singly linked	forward only	simplest stacks/queues
Doubly linked	both directions (prev + next)	undo/redo, LRU caches, dequeues
Circular	wraps at the end, no NULL	round-robin scheduling, playlists
Multilevel / Skip list	skips ahead, drops down	ordered dicts without a balanced tree

Skip lists: randomized levels

Each node gets a random level on insert (level 1 w.p. $1/2$, level 2 w.p. $1/4$, ...). Search starts at the top-left node, moves right while `next key` $\leq$ target, drops a level when it would overshoot. Expected search cost $O(\log n)$ — the extra levels act as self-organising "express lanes" over the base list (Handbook §13.3).

JOSEPHUS ELIMINATION

A classic circular-list application: n people in a circle, every k -th is eliminated. Implemented by stepping a pointer forward k times (skipping removed nodes) and unlinking where it lands.

Stacks

LIFO — Last In, First Out

Insertion and deletion happen at one end only, the top. Deletion always removes the most recently inserted element (Handbook §2.4). `push(x)` · `pop()` · `peek()/top()` · `isEmpty()` — all $O(1)$. A stack is a discipline, not a new memory layout: restrict a list/array to end-only operations.

	Array-based	Linked-list based
push / pop	$O(1)$	$O(1)$
Overflow risk	yes — fixed capacity	no (until memory runs out)
Extra space	none	1 pointer / node
Cache behaviour	contiguous, cache-friendly	scattered, more cache misses

Array implementation, in brief

- top starts at -1 (empty). push: `stack[++top] = x`. pop: `x = stack[top--]`.
- Overflow: push when `top == size-1`. Underflow: pop when `top == -1`.
- Linked-list stack: push/pop act only on the front node — no shifting, no fixed capacity.

Where stacks show up

- Expression evaluation — infix $\leftrightarrow$ postfix $\leftrightarrow$ prefix conversion and evaluation.
- Balanced-symbol checking — push openers, pop and match on closers; stack must end empty.
- The function call stack — each call pushes an activation record; each return pops one. This is why recursion works: every call gets its own frame, stacked LIFO.
- Depth-first search, undo/redo, browser back history.

Infix $\rightarrow$ postfix (shunting-yard idea)

Numbers go straight to output. Before pushing operator `t`, pop and emit every stacked operator with $\geq$ precedence than `t` (strictly greater, or equal and left-*assoc* — $^$ is the exception, right-*associative*). `(` pushes as a wall; `)` pops/emits to the matching `(`.

3 + 4 * 2 - 1 → 3 4 2 * + 1 -

Queues

FIFO — First In, First Out

Insert at the rear, delete from the front. Deletion always removes the earliest-inserted element (Handbook §2.4). enqueue(x) · dequeue() · peek()/front() · isEmpty(). Needs two tracked ends — front and rear — unlike a stack's single top.

The circular-array fix

A naive array queue drifts rightward as you enqueue/dequeue: freed slots on the left sit wasted even though the array is not full (Handbook §2.4.2, Fig. 2.12). Fix: treat the array as a ring, index size−1 wraps to 0 via % size.

- Convention: front == rear means empty. This sacrifices one slot — a size-slot array holds at most size−1 elements — so front reaching rear after an insert unambiguously means full.
- Without sacrificing that slot, "full" and "empty" look identical (front == rear in both).

	Circular array	Linked list
enqueue / dequeue	O(1)	O(1)
Overflow risk	yes — size−1 usable slots	no (until memory runs out)
Typical use	known/bounded size, ring buffers	unknown or unbounded size

Deque and priority queue

- Deque (double-ended queue): addFront, addRear, delFront, delRear — all O(1). A stack is a deque used at one end; a plain queue is a deque with adds at one end and deletes at the other.
- Priority queue: removes the highest- (or lowest-) priority element next, not the earliest arrival. A sorted array/list gives O(n) insert, O(1) remove-highest; a heap (later topic) does both in O(log n).

Where queues show up

- Job/task scheduling, print spoolers, call-centre request handling (FCFS).
- Breadth-first search — a queue of frontier nodes, explored level by level.
- Buffering (keystrokes, network packets) and round-robin CPU scheduling.

Compare & Review

Every linear structure, side by side

Structure	Access	Search	Insert front	Insert end	Delete
Static array	$O(1)$	$O(n)$ / $O(\log n)$ sorted	$O(n)$ shift	$O(1)$ if room	$O(n)$ shift
Dynamic array	$O(1)$	$O(n)$ / $O(\log n)$ sorted	$O(n)$ shift	$O(1)$ amortized	$O(n)$ shift
Singly/doubly/circular list	$O(n)$	$O(n)$	$O(1)$	$O(1)$ with tail ptr	$O(1)$ once located
Skip list	$O(n)$	$O(\log n)$ expected	$O(1)$	$O(\log n)$ expected	$O(\log n)$ expected
Stack (array / list)	—	—	push/pop $O(1)$	—	—
Queue (array / list)	—	—	enqueue/dequeue $O(1)$	—	—

"Once located" = after already holding a pointer to the node; walking there from the head is still $O(n)$.

Which structure should I reach for?

Fast random access by index	→ Array (static if size known, dynamic otherwise)
Frequent insert/delete in the middle, no index access needed	→ Linked list
Need to walk backward as easily as forward	→ Doubly linked list
Round-robin / wrap-around access, no fixed "end"	→ Circular linked list
Ordered search near $O(\log n)$ without a balanced tree	→ Skip list
One end only, last-in-first-out	→ Stack
Add one end, remove the other, first-in-first-out	→ Queue
Both ends active, or priority beats arrival order	→ Deque, or priority queue

Tree Basics

What is a tree?

A non-linear, hierarchical structure: nodes joined by edges so there is exactly one path between any two nodes. Recursive view: a tree is a root plus zero or more subtrees, each itself a tree. A tree with n nodes has exactly $n-1$ edges.

QUICK TEST

connected ✓ no cycle ✓ $n-1$ edges ✓ — if any one fails, it is not a tree.

Vocabulary

Term	Meaning
Root	top node, has no parent
Parent / child	immediate predecessor / successor
Sibling	nodes with the same parent
Leaf (external)	node with no children
Internal node	node with at least one child
Ancestor	on the path from root to the node (not itself)
Descendant	reachable going downward from the node
Degree of a node	number of children it has
Depth / level of a node	edges from the root down to it — root is depth 0
Height of a node	edges on the longest path down to a leaf — a leaf has height 0
Height of the tree	height of the root
Subtree	a node plus all its descendants

WATCH THE CONVENTION

This deck: root depth 0, leaf height 0, empty tree height -1 . Handbook Ch.3: root at level 0, height = max level. Handbook Ch.10: height counts empty (external) nodes too. Read which convention a formula uses before you apply it.

Storing a general tree

- List representation: write the root, then the list of its subtrees in brackets — (A (B (E, F), C)). Grows dynamically, handles any number of children per node, no wasted cells for sparse trees.
- Left-Child Right-Sibling (LCRS): every node keeps just two pointers — left = first child, right = next sibling. The result is an ordinary binary tree, so binary-tree code can be reused. Saves memory and needs no known maximum branching factor (Handbook Lemma 3.1: one-to-one correspondence between forests and binary trees).

Types of trees by maximum children

Type	Max children	Examples
Binary tree	≤ 2 (left, right)	BST, AVL, Red-Black, Splay, heaps
Ternary tree	≤ 3 (left, mid, right)	ternary search trees, some game trees
N-ary (generic) tree	$\leq N$	B-trees, B+ trees, tries

Binary Trees & Traversals

Shapes

Shape	Rule
Full	every node has 0 or 2 children
Complete	all levels full except maybe the last, filled left → right
Perfect	all internal nodes have 2 children; all leaves on one level
Skewed	every node has one child — a chain, height $n-1$

Counting rules

- At level l : at most 2^l nodes. At height h : at most $2^{h+1} - 1$ nodes (perfect tree).
- n nodes → height at least $\text{floor}(\log_2 n)$.
- $n_0 = n_2 + 1$ — leaves are always one more than two-child nodes (proof: count edges two ways — $n-1$ total, and $n_1 + 2 \cdot n_2$ from the parent side).
- Number of distinct BSTs on n keys: $C(2n, n)/(n+1)$ — the Catalan numbers: 1, 2, 5, 14, 42, 132, 429 for $n = 1..7$. Counts explode fast — most shapes are tall and thin, which motivates balancing.

Storing a binary tree: array or pointers

Array (1-indexed), for the node at index i : parent = $\text{floor}(i/2)$; left child = $2i$; right child = $2i+1$. Compact for complete trees, wastes cells for skinny ones (a right-skewed tree of n nodes can need $2^n - 1$ cells). Pointers: one node per key, grows/shrinks freely — the usual choice for BSTs.

Four traversals

Traversal	Order	Structure used
Pre-order	Root · Left · Right	stack (DFS)
In-order	Left · Root · Right	stack (DFS) — sorted order on a BST
Post-order	Left · Right · Root	stack (DFS)
Level-order	top→bottom, left→right	queue (BFS)

Traversal	Time	Extra space
Pre / In / Post-order	$O(n)$	$O(h)$ recursion stack
Level-order	$O(n)$	$O(\text{width})$ queue

Rebuilding a tree from two traversals

- Pre-order's first key is the root. Find it in the in-order list: everything left of it is the left subtree, everything right is the right subtree — recurse.
- Pre+In (or Post+In) rebuild the tree uniquely. Pre+Post do NOT — e.g. "A with one child B" fits both pre-order AB and post-order BA as a left child or a right child.

EXAM TRICK FOR PRE/IN/POST

Dot on the LEFT of each node = pre-order; BELOW = in-order; RIGHT = post-order. Walk around the tree hugging the edges from the root; write a node down each time you pass its dot.

Binary Search Trees

The BST rule

left subtree < node < right subtree, for every node

- Both subtrees must themselves be BSTs — checking only "left child < node < right child" is NOT enough; every key in the whole left subtree must be smaller than the node.
- Keys are unique. In-order traversal = sorted order — the fast validity test: run an in-order walk; if the output is strictly increasing, it is a BST.
- Height: about $\log_2 n$ if balanced, up to $n-1$ if skewed.

Search, insert, delete

- Search: compare with the current node; smaller $\rightarrow$ left, larger $\rightarrow$ right; empty pointer means not found. Cost = nodes on the path = $O(h)$.
- Insert: same walk as search, until it reaches an empty (NULL) pointer — attach the new node there. It is always a new leaf. Never rearranges existing nodes.
- Delete — 3 cases: leaf (cut the link) · one child (child moves up into its place) · two children (replace the key with its in-order successor or predecessor, then delete that node — which is now case 1 or 2).

INSERTION ORDER MATTERS

Sorted input turns a plain BST into a linked list — search/insert/delete all cost $O(n)$. Balanced trees (AVL, Red-Black, Splay) are the cure.

Operations at a glance

Operation	How	Time
Search / Insert / Delete	one root-to-leaf path (+ local repair)	$O(h)$
Minimum / Maximum	keep going left / right	$O(h)$
Successor of x	leftmost node in x's right subtree	$O(h)$
In-order traversal	visits keys in sorted order	$O(n)$
Range search [k1,k2]	in-order walk, skip subtrees outside range	$O(h + \text{output})$

Here h is the tree's height: $O(\log n)$ balanced, $O(n)$ skewed — " $O(h)$ " means good days and bad days.

Strengths, weaknesses, uses

Strengths	Weaknesses
Search/insert/delete in $O(h)$ · sorted order always · easy successor/predecessor/range queries	Not self-balancing — sorted input gives $O(n)$ · extra memory for two pointers/node · poor cache behaviour

AVL Trees I — Balance & Rotations

Balance factor

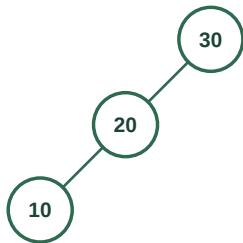
$$BF(x) = \text{height}(\text{left}) - \text{height}(\text{right})$$

- An empty subtree has height -1 . AVL rule: $|BF| \leq 1$ at every node.
- Named after Adelson-Velsky and Landis (1962) — the first balanced BST (Handbook 10.4.1).
- Guarantee: $\text{height} \leq 1.44 \cdot \log_2(n+1)$, so search, insert and delete are all $O(\log n)$.

Rotation: the one repair tool

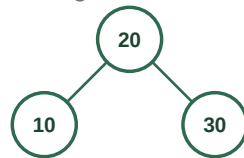
A single right or left rotation at a node lifts its heavy child up. Only the "middle" subtree changes parent — three pointers change, so a rotation costs $O(1)$. In-order is preserved, so the BST rule is never broken. Right and left rotations undo each other.

LL case — unbalanced



→

after right rotation at 30



AVL tree = BST + $BF \in \{-1, 0, +1\}$

An in-order traversal is still sorted, because an AVL tree is still a BST — the balance rule only constrains its shape.

Four ways an AVL tree goes out of balance

Case	New key went into...	Fix
LL (straight)	left subtree of the left child	one right rotation at the node
RR (straight)	right subtree of the right child	one left rotation at the node
LR (bend)	right subtree of the left child	left rotate the child, then right rotate the node
RL (bend)	left subtree of the right child	right rotate the child, then left rotate the node

Rule of thumb: straight line → ONE rotation. Bend → TWO rotations.

AVL insertion, step by step

1. Insert like a BST — walk down, compare, attach the new key as a leaf.
2. Walk back up from the new leaf toward the root, recomputing heights/BFs.
3. Find the lowest ancestor with $BF = \pm 2$ — that is the one to fix.
4. Rotate: LL → right · RR → left · LR → left+right · RL → right+left.

AVL Trees II — Insertion, Deletion & Summary

Insertion vs. deletion

- After an INSERTION, one rotation (single or double) restores balance for the WHOLE tree: $O(1)$ structural change, $O(\log n)$ total including the walk up.
- After a DELETION, the fix at one node can shorten its subtree and unbalance the parent — so you may need to rotate at every level on the way up: up to $O(\log n)$ rotations.
- Deleting a two-child node: use its in-order successor or predecessor as in a plain BST, then rebalance starting from the spot where a node was actually removed.
- Searching an AVL tree is ordinary BST search: $O(\log n)$ guaranteed (never degrades).

How tall can an AVL tree be? (Handbook 10.4.1)

Fewest nodes $N(h)$ in an AVL tree of height h : a minimal tree has one subtree of height $h-1$ and the other of height $h-2$ (the shortest the rule allows), plus the root.

$$N(h) = N(h-1) + N(h-2) + 1$$

h	0	1	2	3	4	5	6	7
N(h)	1	2	4	7	12	20	33	54

These are Fibonacci numbers — $N(h) = F(h+3) - 1$ — which grow like ϕ^h with $\phi \approx 1.618$. So $n \geq \phi^h$ and the height is at most $\log_\phi(n) \approx 1.44 \log_2 n$.

AVL trees: summary

Operation	Time	Rotations
Search	$O(\log n)$, always	—
Insert	$O(\log n)$	at most 1 (single or double)
Delete	$O(\log n)$	up to $O(\log n)$

Height bound: $h \leq 1.44 \cdot \log_2 n$

Strengths	Weaknesses
Guaranteed $O(\log n)$ for every op · shortest trees → fastest searches · predictable, good for real-time use	Harder to code than a plain BST · more rotations on updates than Red-Black · extra field per node (height or BF)

FIX THE LOWEST VIOLATION

Always repair the LOWEST ancestor with $|BF| = 2$ on the path from the new/removed key to the root — never a node further up, even if it also looks unbalanced at that moment.

Red-Black Trees I — Rules & Insertion

The five Red-Black rules

- 1. Every node is red or black.
- 2. The root is black.
- 3. Every leaf (an empty NIL pointer) is black.
- 4. No double red: a red node never has a red child.
- 5. Black height: every path from a node down to a NIL leaf has the same number of black nodes.

Why colours keep the tree short

- Rule 5: every root-to-leaf path has the same number of blacks (say bh). Rule 4: no two reds in a row, so at most every second node is red — any path has at most $2 \cdot bh$ nodes.
- So the longest path is at most twice the shortest. A subtree with black height bh holds $\geq 2^{bh} - 1$ nodes, so $bh \leq \log_2(n+1)$.

height $\leq 2 \cdot \log_2(n + 1)$

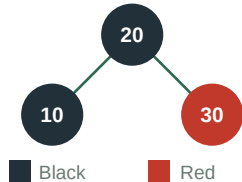
Slightly taller than AVL's $1.44 \cdot \log_2 n$ — but keeping the rules needs fewer rotations.

Insertion: the decision flow

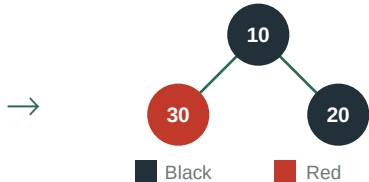
Insert X like a BST and colour it RED. G = grandparent, P = parent, U = uncle (parent's sibling — a missing/NIL uncle counts as black).

Case	Condition	Fix
Root	X is the root	colour X black — done
—	parent P is black	nothing to fix — done
1 — recolour	uncle U is red	P, U $\rightarrow$ black; G $\rightarrow$ red; treat G as new X, repeat
2 — line	uncle black, X/P/G straight	rotate G away from X, swap colours of P and G — done
3 — triangle	uncle black, X/P/G bent	rotate P to make a line, then do Case 2

insert 30: red-red (20–30)



Case 2: rotate left at G=10, swap colours



Red-Black Trees II — Deletion & Summary

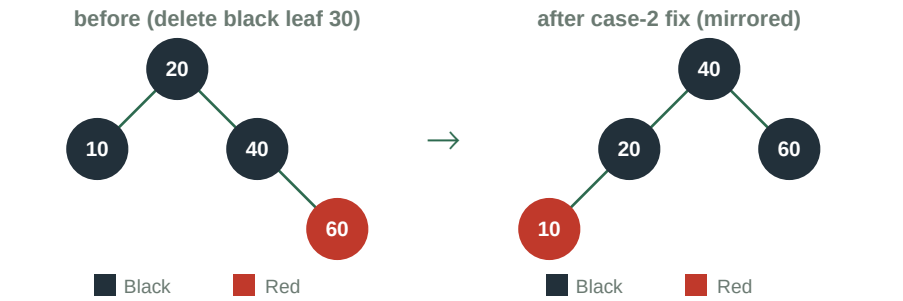
Deletion: fixing a "double black"

BST-delete as usual. If the node that physically leaves (or its replacement) is red, recolour it black and stop. If it was black, every path through that spot lost one black — the replacement becomes "double black" (DB). Fix DB using its sibling W, parent P, and W's children (the nephews). Cases below are for DB as a LEFT child; mirror for a right child.

Case	Sibling W	Nephews	Then
1	red	both black	W→black, P→red, rotate P toward DB — becomes case 2/3/4
2	black	both black	W→red; push extra black up to P — may repeat higher up
3	black	near red, far black	swap colours of W & red child, rotate W away from DB — becomes case 4
4	black	far child red	W takes P's colour; P & far child→black; rotate P toward DB — DONE

Near nephew = W's child on the same side as DB; far nephew = the other side.

Deletion example: delete 30



X's sibling (10) is black with two black (NIL) children → case 2 (mirrored): 10→red, extra black moves up to P=20; P is now recoloured into the black 40 root's left side. Every path from 40 to a NIL still has 2 black nodes.

Red-Black trees: summary

Operation	Time	Rotations
Search	$O(\log n)$	—
Insert	$O(\log n)$	at most 2 (+ recolouring)
Delete	$O(\log n)$	at most 3 (+ recolouring)

Height bound: $h \leq 2 \cdot \log_2(n+1)$ · Where you find them: C++ map/set, Java TreeMap, Linux CFS scheduler, file-system structures.

Splay Trees

The accessed node goes to the root

Self-adjusting BST: after every search, insert or delete, the touched node is rotated up until it is the root. No extra data — no heights, colours or balance factors stored. Recently used keys stay near the top (locality of reference), so repeating an access is cheap.

Three cases cover every splay step

Case	Shape	Move
Zig / Zag	x's parent p is the ROOT	one rotation: x over p (only as the last step)
Zig-Zig / Zag-Zag	x and p both left (or both right) children	rotate p over g FIRST, then x over p
Zig-Zag / Zag-Zig	x and p on opposite sides	rotate x over p, then x over g

ORDER MATTERS

Rotating only x twice ("simple move-to-root") does NOT shorten the path the way zig-zig does — rotate the PARENT first.

Every operation is a splay plus a little extra

- Search(x): ordinary BST search. Found → splay it to the root. Not found → splay the LAST node visited on the search path.
- Insert(x): ordinary BST insertion as a leaf, then splay the new node to the root.
- Delete(x): splay x to the root and remove it, leaving trees L and R. If L is empty, R is the answer; else splay the maximum of L, then hang R as its right child.



Why O(log n)? Amortized cost

Potential method: $\text{rank}(x) = \log_2(\text{size of } x\text{'s subtree})$, $\Phi(T) = \sum \text{rank}(x)$. amortized cost = actual cost + (Φ after – Φ before). Access lemma: $\text{splay}(x)$ has amortized cost $\leq 3 \cdot (\text{rank}(\text{root}) - \text{rank}(x)) + 1$.

m operations, $\leq n$ keys: $O((m + n) \log n)$ total time

Good at / bad at

Good for	Watch out
A few keys accessed again and again · caches, memory managers · garbage collection, link-cut trees	A single operation can cost $O(n)$ — unsuitable for real-time systems · every read restructures the tree (more writes)

Compare & Review

AVL vs Red-Black vs Splay

	AVL	Red-Black	Splay
Balance info	height/BF per node	1 colour bit per node	none
Height bound	$\leq 1.44 \log_2 n$	$\leq 2 \log_2(n+1)$	up to $n-1$ (amortized $O(\log n)$)
Search	$O(\log n)$, fastest	$O(\log n)$	amortized $O(\log n)$
Insert	≤ 1 rotation	≤ 2 rotations + recolour	amortized $O(\log n)$, many rotations
Delete	up to $O(\log n)$ rot.	≤ 3 rotations + recolour	amortized $O(\log n)$
Worst single op	$O(\log n)$	$O(\log n)$	$O(n)$
Best when	lookups dominate	general-purpose, mixed updates	a few keys reaccessed often
Real-world	Multiset/Map/Set	DB txns; C++ map, Java TreeMap	caches, garbage collection

RULE OF THUMB

Hard guarantees, fast lookups → AVL. Cheap updates with guarantees → Red-Black. Skewed access pattern, average time is enough → Splay.

One-page formula sheet

Topic	Key facts
Tree basics	root depth 0, leaf height 0, empty tree -1 · n nodes → $n-1$ edges · level $l \leq 2^l$ nodes · height $h \leq 2^{h+1}-1$ nodes · min height for n nodes: $\text{floor}(\log_2 n)$ · $n_0 = n_2 + 1$ · BSTs on n keys: $C(2n, n)/(n+1) \rightarrow 1, 2, 5, 14, 42$ · array(1-based): parent $\text{floor}(i/2)$, children $2i, 2i+1$ · Pre=RLR · In=LRR · Post=LRR · Level=queue
BST & AVL	BST: left<node<right; in-order sorted; $O(h)$ · delete: 0/1 child, or copy in-order successor/predecessor · BF=height(left)-height(right), AVL needs $ BF \leq 1$ · fix the LOWEST $ BF =2$ node: LL→right · RR→left · LR→left,right · RL→right,left · fewest AVL nodes: $N(h)=N(h-1)+N(h-2)+1 \rightarrow 1, 2, 4, 7, 12, 20, 33, 54$ · height $\leq 1.44 \log_2 n$; insert ≤ 1 rotation, delete $\leq O(\log n)$
Red-Black & Splay	RB: root black · NIL black · no red-red · equal black count every path · insert RED; uncle red→recolour; uncle black+line→rotate G+swap colours; triangle→rotate P first · delete black node→double black: cases 1–4 by sibling colour/nephews · height $\leq 2 \log_2(n+1)$ · Splay: zig · zig-zig (parent first) · zig-zag; failed search splays the last node · delete: splay x, remove, splay max(L), attach R · amortized = actual + potential change → $O((m+n) \log n)$